

Redes de Computadores

Computer Networks

Lab 02 – Stream oriented network programming with TCP sockets

Summary

- Client/Server model
- Example (Echo Client and Server)
- Assignments

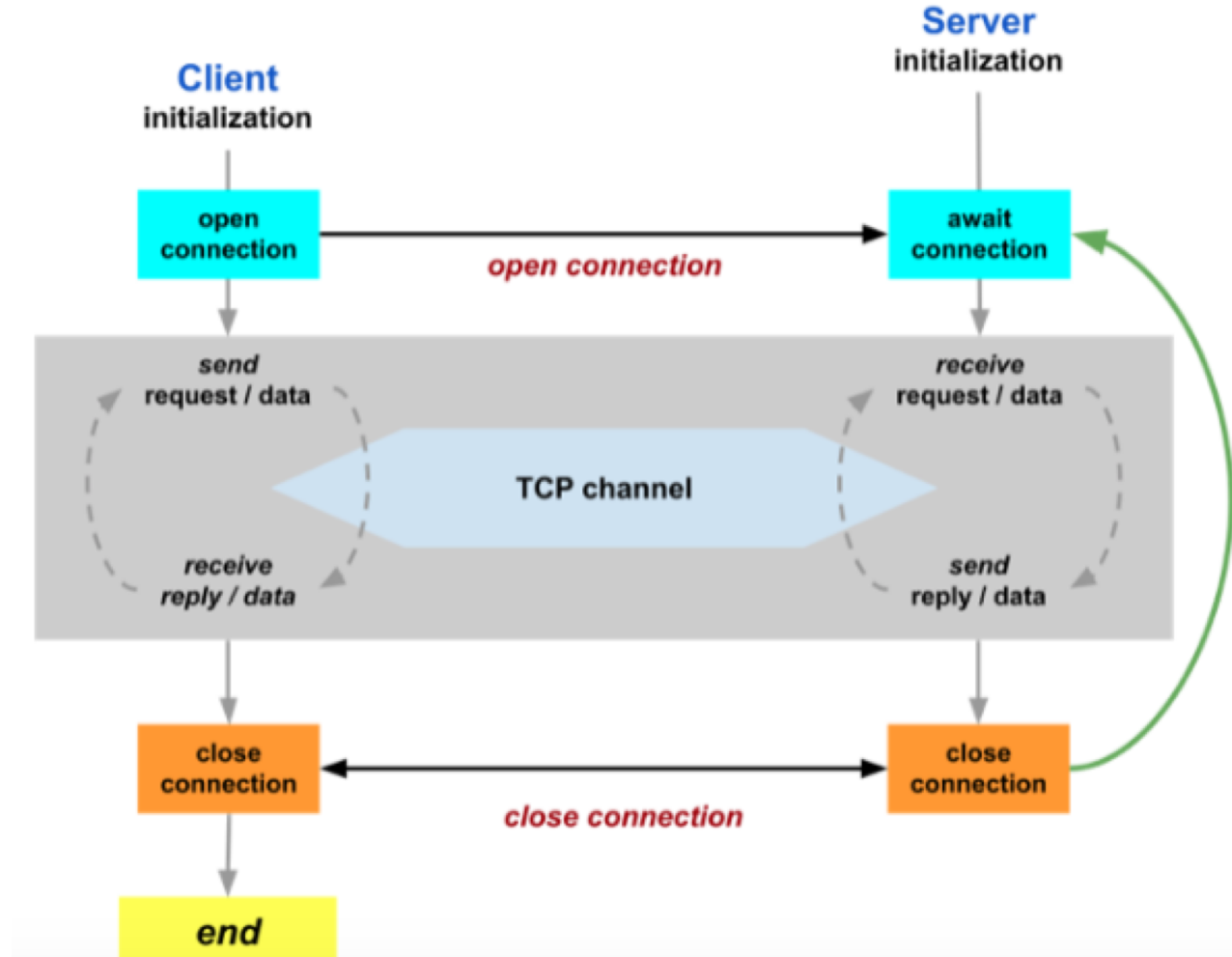
Client/Server Model

☐ Two autonomous components:

- **server** - first to run and usually always running
- **client** - usually started by user to request a service

☐ TCP follows the same abstract client/server model that characterizes UDP communication

Client/Server execution diagram



TCP Channel

- **Reliable** connection between two processes;
- Usually **initiated** by the client, addressed to a machine and port, where the server **already** awaits connection requests;
- Enables a continuous and bidirectional flow of data (bytes) between the two processes;
- Closed at the request of **either** the client or the server;
- Both the client and server **are aware** when a connection is established or broken/closed.

TCP oriented communication

- Communication is based on **sockets**
 - TCP sockets for accepting connections (server)
 - TCP sockets for data exchange (client and server)
- Addresses identify connection endpoints (origin and destination processes/sockets)
 - **Host** (IP Address) 10.1.233.67, 127.0.0.1, 192.168.1.1, etc.
 - **Port** (16 bits) 80, 443, 8000, 8080, etc.

TCP Client

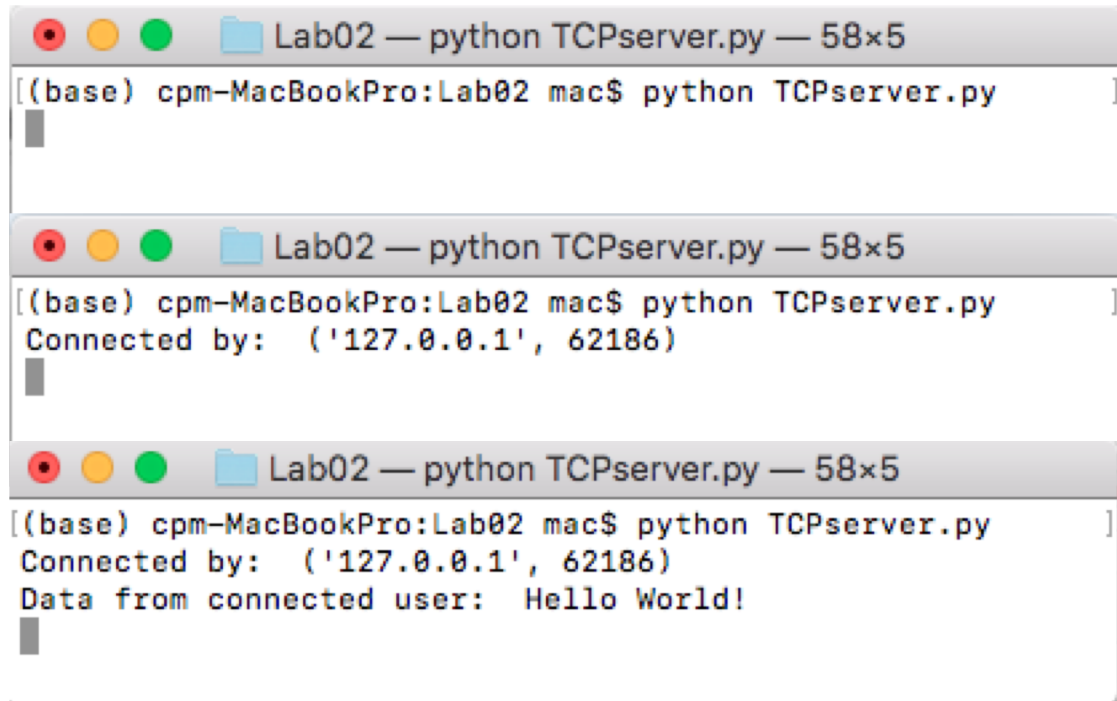
```
TCPclient.py
TCPclient.py > No Selection
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  @author: cpm
5  """
6  from socket import *
7
8  serverName = "localhost"           # server name
9  serverPort = 12000                 # socket server port number
10 sockBuffer = 2048                  # socket buffer size
11
12 def main():
13     clientSocket = socket(AF_INET, SOCK_STREAM)    # create TCP socket
14     clientSocket.connect((serverName, serverPort)) # open TCP connection
15
16     sentence = input("Input lowercase sequence: ") # take input
17
18     clientSocket.send(sentence.encode())           # send user's sentence
19                                                    # over TCP connection
20
21     modifiedSentence = clientSocket.recv(sockBuffer).decode() # receive response
22     print("Message received: ", modifiedSentence)
23
24     clientSocket.close()                          # close TCP connection
25
26 main()
27
```

TCP Server

```
TCPserver.py
3 """
4 @author: cpm
5 """
6 from socket import *
7
8 serverPort = 12000
9 sockBuffer = 2048
10
11 def main():
12
13     serverSocket = socket(AF_INET, SOCK_STREAM) # create TCP welcoming socket
14     serverSocket.bind(("", serverPort))
15
16     serverSocket.listen(1) # begin listening for incoming TCP requests
17     print("Server is running")
18
19     while True:
20         connSocket, addr = serverSocket.accept() # waits for incoming requests:
21                                                  # new socket created on return
22         print("Connected by: ", str(addr))
23
24         sentence = connSocket.recv(sockBuffer).decode() # read a sentence of bytes
25                                                         # received from client
26
27         print("Data from connected user: ", sentence) # display received sentence
28
29         sentenceUpper = sentence.upper() # convert to upper case
30
31         connSocket.send(sentenceUpper.encode()) # send modified sentence over
32                                                 # TCP connection
33
34         connSocket.close() # close TCP connection:
35                             # the welcoming socket continues
36
37 main()
```


Assignment 1

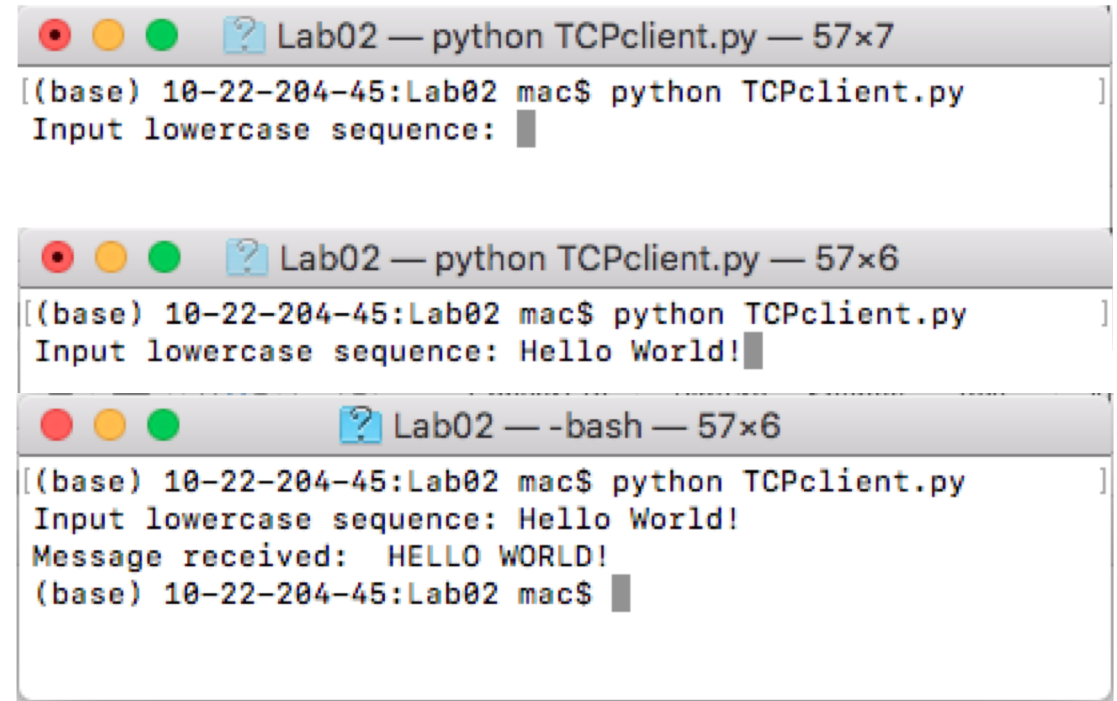
- Get the source code of *TCPclient.py* and *TCPserver.py* available from CLIP.
- Study the source code and run it.
 - You can invoke the Python interpreter in a shell.



```
Lab02 — python TCPserver.py — 58x5
[(base) cpm-MacBookPro:Lab02 mac$ python TCPserver.py]

Lab02 — python TCPserver.py — 58x5
[(base) cpm-MacBookPro:Lab02 mac$ python TCPserver.py]
Connected by: ('127.0.0.1', 62186)

Lab02 — python TCPserver.py — 58x5
[(base) cpm-MacBookPro:Lab02 mac$ python TCPserver.py]
Connected by: ('127.0.0.1', 62186)
Data from connected user: Hello World!
```



```
Lab02 — python TCPclient.py — 57x7
[(base) 10-22-204-45:Lab02 mac$ python TCPclient.py]
Input lowercase sequence:

Lab02 — python TCPclient.py — 57x6
[(base) 10-22-204-45:Lab02 mac$ python TCPclient.py]
Input lowercase sequence: Hello World!

Lab02 — -bash — 57x6
[(base) 10-22-204-45:Lab02 mac$ python TCPclient.py]
Input lowercase sequence: Hello World!
Message received: HELLO WORLD!
(base) 10-22-204-45:Lab02 mac$
```

Assignment 2 – Adding two numbers

In this assignment, you'll write a client that will use TCP sockets to communicate with a server that you will also write. Here's what your client and server should do:

Client

1. Accept a name and an integer between 1 and 100 from the keyboard;
2. Create a TCP socket and open a connection to the port agreed with the server;
3. Send to the server: (i) the entered name and (ii) the entered integer value and then wait for a server reply;
4. When the client receives the reply from the server it should process it and then display:
 - its name and the server's name;
 - Its integer value, the server's integer value and the sum of both;
 - The client then terminates after closing any created sockets. As a note (and as a check that you are doing things correctly) you should make sure for yourself that the values and the sums are correct!

Server

1. Create a string containing a name (e.g., "Server of John Q. Smith") and a random number;
2. Create a TCP socket, bind it to the agreed port, and wait for a connection;
3. On accepting a client connection, the server should:
 - Display the client's received name and the server's name;
 - Display the client's number, its number, and the sum of those numbers;
 - Reply to the client with the server name and the server number;
 - If your server receives an integer value that is out of range, it should terminate after releasing any created sockets. You can use this to shut down your server.

Assignment 3 – File Transfer

You should write a file transfer program where:

- The server is invoked with the following command line:

```
>python TCPserver.py port
```

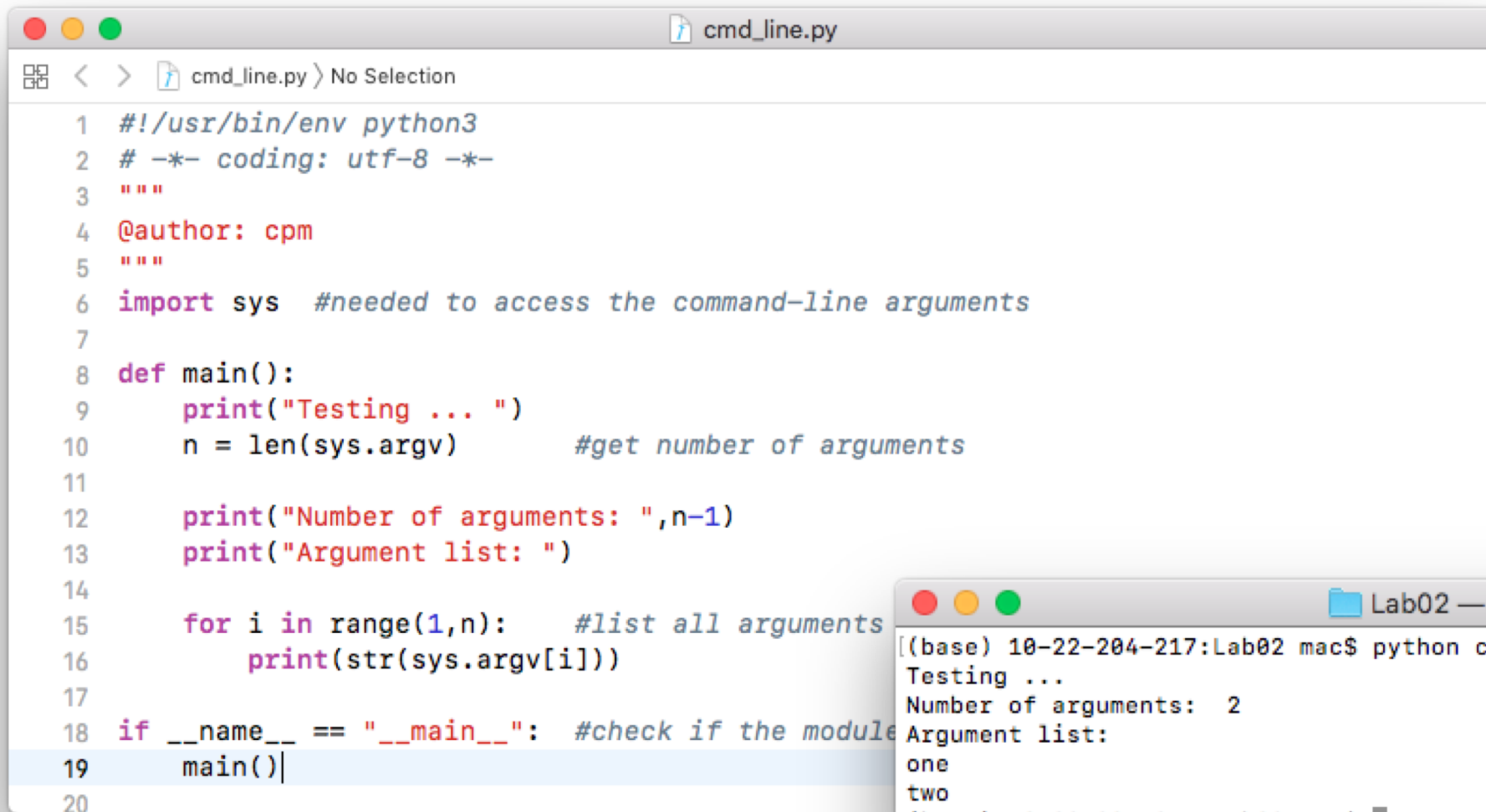
- The client is invoked with the following command line:

```
>python TCPclient.py server:port:fNameInServer fNameInClient
```

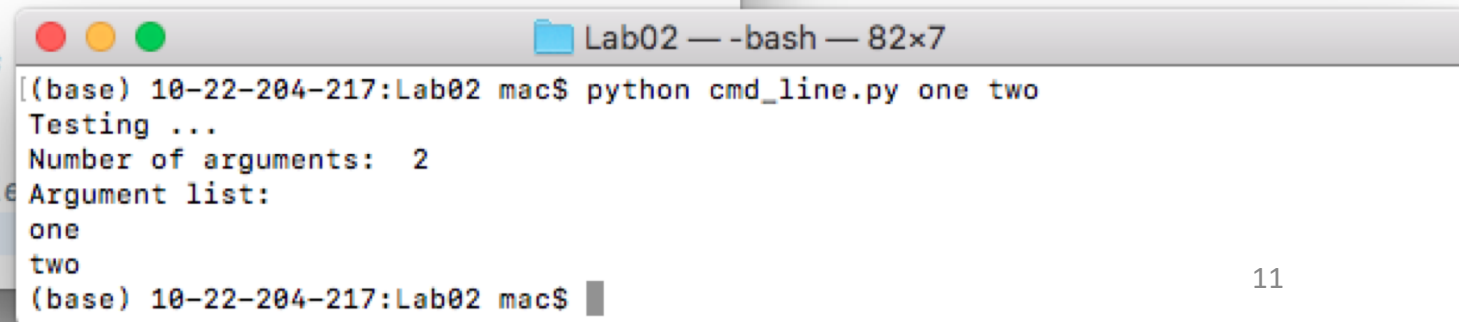
- The client connects to the server
- The client sends the file name to be transferred
- The server sends the file to the client in 1024 bytes blocks
- Client receives the blocks, sent by the server, and writes them to its local disk.
- Compare the contents of the two files (windows: fc command; linux and macOS: diff command).

Assignment 3

Python - command line commands



```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 @author: cpm
5 """
6 import sys #needed to access the command-line arguments
7
8 def main():
9     print("Testing ... ")
10     n = len(sys.argv)      #get number of arguments
11
12     print("Number of arguments: ",n-1)
13     print("Argument list: ")
14
15     for i in range(1,n):    #list all arguments
16         print(str(sys.argv[i]))
17
18 if __name__ == "__main__": #check if the module is being run directly
19     main()
20
```



```
Lab02 — -bash — 82x7
((base) 10-22-204-217:Lab02 mac$ python cmd_line.py one two
Testing ...
Number of arguments:  2
Argument list:
one
two
(base) 10-22-204-217:Lab02 mac$
```

```
fileCopy.py

1  #!/usr/bin/env python3
2
3  SIZE = 1024
4
5  def main():
6      fName = input("File to copy: ")
7      try:
8          f = open(fName, "rb")
9      except:
10         data = "File " + fName + " not found"
11         exit(1)
12
13     fName2 = "Copy"
14     fC = open(fName2, "wb")
15     data = f.read(SIZE)
16     print("Copying ")
17     print(".",end='')
18     while (data):
19         fC.write(data)
20         print(".",end='')
21         data = f.read(SIZE)
22     f.close()
23     fC.close()
24     print("\nFile",fName,"has been copied")
25
26 main()
```

Python – copy a file

Assignment 3

Python - command line commands (Spyder)

Run -> Configuration per file

